WEEK2 - TASKS

Cognizant-upGrad Training Program uGE_DotNet FSD(python) Weekly Submission Document

Name: Golla Amulya

Batch Name:B1_Python

Course:uGE DotNet FSD (python)
Training Duration: 16th Feb 2026-26th may 2026
Faculty: Kunal

23/02/2026 **BOOTSTRAP**

Question-1:
Create a responsive Product section with 4 columns on large screens, 2 columns on medium screens, and 1 column on small screens.

```
<!DOCTYPE html>
<html>
<head>
  <title>Responsive Products</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>

<div class="container my-5">

  <h2 class="text-center mb-4">Our Products</h2>

  <div class="row g-3">


    <div class="col-lg-3 col-md-6 col-12">
      <div class="p-3 border rounded text-center shadow-sm">
        <h5>Mobile Phone</h5>
        <p>₹15,000</p>
        <button class="btn btn-primary">Buy Now</button>
      </div>
```

```
    </div>


    <div class="col-lg-3 col-md-6 col-12">
      <div class="p-3 border rounded text-center shadow-sm">
        <h5>Laptop</h5>
        <p>₹45,000</p>
        <button class="btn btn-success">Buy Now</button>
      </div>
    </div>


    <div class="col-lg-3 col-md-6 col-12">
      <div class="p-3 border rounded text-center shadow-sm">
        <h5>Headphones</h5>
        <p>₹2,000</p>
        <button class="btn btn-warning">Buy Now</button>
      </div>
    </div>


    <div class="col-lg-3 col-md-6 col-12">
      <div class="p-3 border rounded text-center shadow-sm">
        <h5>Smart Watch</h5>
        <p>₹5,000</p>
        <button class="btn btn-danger">Buy Now</button>
      </div>
    </div>

  </div>

</div>

</body>
</html>
```

Question-2
Create a Bootstrap Navbar with: Brand name:Myshop

```
<!DOCTYPE html>

<html>
```

```
<head>

  <title>MyShop Navbar</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark px-3">

  <a class="navbar-brand fw-bold" href="#">MyShop</a>

  <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#menuArea">

    <span class="navbar-toggler-icon"></span>

  </button>

  <div class="collapse navbar-collapse" id="menuArea">

    <ul class="navbar-nav mx-auto mb-2 mb-lg-0">

      <li class="nav-item">

        <a class="nav-link active" href="#">Home</a>

      </li>



      <li class="nav-item">

        <a class="nav-link" href="#">Products</a>

      </li>

      <li class="nav-item">

        <a class="nav-link" href="#">About</a>

      </li>
```

```
      <li class="nav-item">

        <a class="nav-link" href="#">Contact</a>

      </li>

    </ul>

    <button class="btn btn-outline-light">Login</button>

  </div>

</nav>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>

</body>

</html>
```

Question-3
Create a "Customer Support" Section using Bootstrap utilities:

```
<!DOCTYPE html>
<html>
<head>
  <title>Customer Support</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>

<!-- Customer Support Section -->
<div class="container my-5">

  <div class="bg-light text-center p-4 border rounded shadow-sm">

    <h3>Customer Support</h3>
    <p>We are here to help you with any issues or questions.</p>
```

```
    <!-- Button to open Offcanvas -->
    <button class="btn btn-info" data-bs-toggle="offcanvas" data-bs-
target="#supportPanel">
      Support Menu
    </button>

  </div>

</div>


<!-- Offcanvas Menu -->
<div class="offcanvas offcanvas-end" tabindex="-1" id="supportPanel">

  <div class="offcanvas-header">
    <h5 class="offcanvas-title">Support Options</h5>
    <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
  </div>

  <div class="offcanvas-body">
    <ul class="list-group">
      <li class="list-group-item">FAQs</li>
      <li class="list-group-item">Contact Support</li>
    </ul>
  </div>

</div>


<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>

</body>
</html>
```

Question-4
Responsive Personal Portfolio Homepage

```
<!DOCTYPE html>
<html>
<head>
  <title>My Portfolio</title>
```

```
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>

<nav class="navbar navbar-expand-lg navbar-light bg-light border-bottom px-3">
  <a class="navbar-brand fw-bold" href="#">Jhansi Portfolio</a>

  <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-
target="#navMenu">
    <span class="navbar-toggler-icon"></span>
  </button>

  <div class="collapse navbar-collapse" id="navMenu">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Skills</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
    </ul>
  </div>
</nav>

<div class="container my-5">

  <div class="row align-items-center text-center text-md-start">

    <div class="col-md-4">
      <img src="https://via.placeholder.com/200" class="img-fluid rounded-circle
border" alt="profile">
    </div>

    <div class="col-md-8 mt-3 mt-md-0">
      <h2>Hello, I'm Amulya</h2>
      <p class="text-muted">Fresher Web Developer | .NET & Python Learner</p>
      <button class="btn btn-primary">Download Resume</button>
    </div>

  </div>

</div>

<div class="container my-5">
```

```
  <h3 class="text-center mb-4">My Skills</h3>

  <div class="row g-3">

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card h-100 shadow-sm">
        <div class="card-body text-center">
          <h5 class="card-title">HTML & CSS</h5>
          <p class="card-text">Basic responsive web design knowledge.</p>
          <button class="btn btn-outline-primary btn-sm">View</button>
        </div>
      </div>
    </div>

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card h-100 shadow-sm">
        <div class="card-body text-center">
          <h5 class="card-title">Bootstrap</h5>
          <p class="card-text">Creating responsive layouts using grid system.</p>
          <button class="btn btn-outline-success btn-sm">View</button>
        </div>
      </div>
    </div>

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card h-100 shadow-sm">
        <div class="card-body text-center">
          <h5 class="card-title">Python / .NET</h5>
          <p class="card-text">Learning backend and programming fundamentals.</p>
          <button class="btn btn-outline-danger btn-sm">View</button>
        </div>
      </div>
    </div>

  </div>

</div>

<div class="container my-5 text-center p-4 bg-light border rounded">

  <h4>Contact Me</h4>
  <p>Email: gollaamulya418@gmail.com</p>
  <p>Phone: 7013931909</p>
```

```
  <button class="btn btn-dark">Send Message</button>

</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>

</body>
</html>
```

Question-5
College Event Management Dashboard

```
<!DOCTYPE html>
<html>
<head>
  <title>College Event Dashboard</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-primary px-3">
  <a class="navbar-brand fw-bold" href="#">College Events</a>

  <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-
target="#collegeMenu">
    <span class="navbar-toggler-icon"></span>
  </button>

  <div class="collapse navbar-collapse" id="collegeMenu">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link active" href="#">Dashboard</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Events</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Students</a></li>
    </ul>
  </div>
</nav>
```

```
<div class="container my-4">

  <div class="row g-3">

    <div class="col-lg-8 col-md-7 col-12">
      <div class="p-3 border rounded bg-light">
        <h4>Upcoming Events</h4>

        <table class="table table-bordered table-sm mt-3">
          <thead class="table-secondary">
            <tr>
              <th>Event Name</th>
              <th>Date</th>
              <th>Department</th>
            </tr>
          </thead>
          <tbody>
            <tr>
              <td>Tech Fest</td>
              <td>12-Mar-2026</td>
              <td>CSE</td>
            </tr>
            <tr>
              <td>Cultural Day</td>
              <td>20-Mar-2026</td>
              <td>Arts</td>
            </tr>
          </tbody>
        </table>

        <button class="btn btn-success" data-bs-toggle="modal" data-bs-
target="#addEventModal">
          Add New Event
        </button>

      </div>
    </div>

    <div class="col-lg-4 col-md-5 col-12">
      <div class="p-3 border rounded bg-white shadow-sm">
        <h5>Quick Event Registration</h5>

        <form class="mt-3">
```

```
        <div class="mb-2">
          <label class="form-label">Event Name</label>
          <input type="text" class="form-control" placeholder="Enter event name"
required>
        </div>

        <div class="mb-2">
          <label class="form-label">Event Date</label>
          <input type="date" class="form-control" required>
        </div>

        <div class="mb-2">
          <label class="form-label">Department</label>
          <select class="form-select">
            <option>CSE</option>
            <option>ECE</option>
            <option>EEE</option>
            <option>MECH</option>
          </select>
        </div>

        <button class="btn btn-primary w-100">Submit</button>

      </form>
    </div>
  </div>

 </div>

</div>

<div class="modal fade" id="addEventModal">
  <div class="modal-dialog modal-md">
    <div class="modal-content">

      <div class="modal-header bg-primary text-white">
        <h5 class="modal-title">Add College Event</h5>
        <button class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
      </div>

      <div class="modal-body">

        <form>
```

```
        <div class="mb-2">
          <label class="form-label">Event Name</label>
          <input type="text" class="form-control is-valid">
        </div>

        <div class="mb-2">
          <label class="form-label">Event Date</label>
          <input type="date" class="form-control is-invalid">
          <div class="invalid-feedback">Please select a date</div>
        </div>

        <div class="mb-2">
          <label class="form-label">Department</label>
          <input type="text" class="form-control">
        </div>
      </form>

    </div>

    <div class="modal-footer">
      <button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
      <button class="btn btn-success">Save Event</button>
    </div>

   </div>
  </div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>

</body>
</html>
```

23/02/2026 **JQUERY**

Question-1:
Student Registration List
File-1:

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title>Student Registration List</title>

    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>

<h2>Student Registration Form</h2>

<input type="text" id="studentName" placeholder="Enter Student Name">
<input type="text" id="rollNo" placeholder="Enter Roll Number">
<button id="addBtn">Add Student</button>

<h3>Student List</h3>
<table border="1" id="studentTable">
    <tr>
        <th>Name</th>
        <th>Roll No</th>
    </tr>
</table>

<script src="StudentRegistrationList.js"></script>

</body>
</html>
```

Question-1:
Student Registration List
File-2:

```
$(document).ready(function () {

    $("#addBtn").click(function () {

        let name = $("#studentName").val();
        let roll = $("#rollNo").val();

        if (name === "" || roll === "") {
            alert("Enter both Name and Roll Number");
            return;
        }
```

```
        let row = `<tr class="studentRow">
                    <td>${name}</td>
                    <td>${roll}</td>
                   </tr>`;

        $("#studentTable").append(row);

        // Clear input
        $("#studentName").val("");
        $("#rollNo").val("");
    });

    // Remove student when clicked
    $("#studentTable").on("click", ".studentRow", function () {
        $(this).remove();
    });

});
```

Question-2
Live Character Counter

```
<!DOCTYPE html>
<html>
<head>
    <title>Live Character Counter</title>

    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>

<body>

<h2>Type Your Message</h2>

<textarea id="msgBox" rows="5" cols="40"></textarea>

<p id="charCount">Characters: 0</p>
<p id="warnMsg"></p>

<script>
$(document).ready(function () {
```

```
    $("#msgBox").keyup(function () {
        let total = $(this).val().length;

        $("#charCount").text("Characters: " + total);

        if (total > 100) {
            $("#warnMsg").text("Limit exceeded!")
                        .css("color", "red");
        }
        else {
            $("#warnMsg").text("")
                        .css("color", "black");
        }
    });

});
</script>

</body>
</html>
```

Question-3
Simple Shopping cart Items.

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            padding: 20px;
        }

        button {
            padding: 6px 10px;
            cursor: pointer;
        }

        ul {
            list-style: none;
            padding: 0;
        }
```

```
        li {
            margin: 5px 0;
            padding: 5px;
            background: #f2f2f2;
            display: flex;
            justify-content: space-between;
            width: 200px;
        }
    </style>
</head>
<body>

<h2>    Simple Shopping Cart</h2>

<!-- Dropdown -->
<select id="itemSelect">
    <option>Apple</option>
    <option>Mango</option>
    <option>Banana</option>
</select>

<button onclick="addItem()">Add to Cart</button>

<h3>Cart Items:</h3>
<ul id="cartList"></ul>

<p>Total Items: <span id="totalCount">0</span></p>

<script>
    let cartCount = 0;

    function addItem() {
        let selectedItem = document.getElementById("itemSelect").value;
        let cartList = document.getElementById("cartList");

        // create list item
        let li = document.createElement("li");
        li.textContent = selectedItem;

        // create remove button
        let removeBtn = document.createElement("button");
        removeBtn.textContent = "Remove";

        removeBtn.onclick = function () {
```

```
            cartList.removeChild(li);
            cartCount--;
            updateCount();
        };

        li.appendChild(removeBtn);
        cartList.appendChild(li);

        cartCount++;
        updateCount();
    }

    function updateCount() {
        document.getElementById("totalCount").innerText = cartCount;
    }
</script>

</body>
</html>
```

24/02/2026 **SQL**

Question-1
Hospital Patient appointment details

```
DROP TABLE IF EXISTS Patientdetails;
CREATE TABLE Patientdetails(
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(50) NOT NULL,
    PatientPhone VARCHAR(15) NOT NULL
);

DROP TABLE IF EXISTS Doctors;
CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY ,
    DoctorName VARCHAR(50) NOT NULL,
    Specialization VARCHAR(50) NOT NULL
);

DROP TABLE IF EXISTS Appointments;
CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY,
```

```
    PatientID INT NOT NULL,
    DoctorID INT NOT NULL,
    AppointmentDate DATE NOT NULL,
    Fees DECIMAL(8,2) NOT NULL,

    FOREIGN KEY (PatientID) REFERENCES Patientdetails(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);


INSERT INTO Patientdetails(PatientID, PatientName, PatientPhone)
VALUES
(1,'KANAKARAJU','9898989898'),
(2,'RAMIREDDY','9879879870'),
(3,'RAMAMURTY','9080908090'),
(4,'NAZINI','9707070909'),
(5,'DWARAKA','9849859860');

INSERT INTO Doctors(DoctorID,DoctorName,Specialization)
VALUES
(01,'Dr AMULYA','Neurology'),
(02,'Dr LAXMI','Dermatology'),
(03,'Dr PRAVEEN','Cardiology'),
(04,'Dr MANJU','Orthopedics'),
(05,'Dr THULASI','ENT Specialist');

INSERT INTO Appointments (PatientID, DoctorID, AppointmentDate, Fees)
VALUES
(1, 01, '2026-02-20', 500.00),
(2, 02, '2026-02-21', 300.00),
(3, 03, '2026-02-22', 700.00),
(4, 04, '2026-02-23', 500.00),
(5, 05, '2026-02-24', 300.00);


SELECT d.DoctorName,d.Specialization,a.AppointmentDate,a.Fees
FROM Appointments a
JOIN Patientdetails p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID;
```

Question-2
(Joins +Filtering+Sorting+Expressions) By Using Company DB

```sql
--DROP TABLE IF EXISTS Departments;

CREATE TABLE Departments (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL
);
--DROP TABLE IF EXISTS Staff;
CREATE TABLE Staff (
   StaffID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Salary DECIMAL(10,2) NOT NULL,
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

--DROP TABLE IF EXISTS Projects;
CREATE TABLE Projects (
   ProjectID INT PRIMARY KEY,
   ProjectName VARCHAR(50) NOT NULL,
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

INSERT INTO Departments(DeptID,DeptName)
VALUES
(1,'IT'),
(2,'FINANCE'),
(3,'HR');


INSERT INTO Staff(StaffID,Name,Salary,DeptID)
VALUES
(1, 'AMULYA', 60000, 1),
(2, 'PRAVEEN', 45000, 2),
(3, 'DEVA', 75000, 1),
(4, 'RAJU', 52000, 3),
(5, 'RANI', 40000, 2);



INSERT INTO Projects(ProjectID,ProjectName,DeptID)
VALUES
(101, 'Website Styling', 1),
(102, 'Employee Portal', 2),
```

```
(103, 'Payment App', 3),
(104, 'Smart shopping', 1);
```

```
SELECT s.StaffID, s.Name, d.DeptName,p.ProjectName
FROM Staff s
JOIN Departments d ON s.DeptID = d.DeptID
JOIN Projects p ON s.DeptID = p.DeptID;
```

```
SELECT s.StaffID,s.Name,d.DeptName,p.ProjectName,s.Salary
FROM Staff s
JOIN Departments d ON s.DeptID = d.DeptID
JOIN Projects p ON s.DeptID = p.DeptID
WHERE s.Salary > 50000
ORDER BY s.Salary DESC;
```

Question-3
(Group By+Having+Subquery)
AShopping app has table:
Orders(OrderID,CustomerName,ProductName,Category,Quantity,Price)

```
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   CustomerName VARCHAR(50) NOT NULL,
   ProductName VARCHAR(50) NOT NULL,
   Category VARCHAR(20) NOT NULL,
   Quantity INT NOT NULL,
   Price DECIMAL(10,2) NOT NULL
);

INSERT INTO
Orders(OrderID,CustomerName,ProductName,Category,Quantity,Price)
VALUES
(1, 'Amulya','Mobile','Electronics', 1, 20000),
(2, 'Manju','Laptop','Electronics', 1, 55000),
(3, 'Praveen','Shirt','Clothing', 3, 800),
(4, 'Thulasi','Rice','Grocery', 2, 1200),
(5, 'Ramya','Headphones','Electronics', 2, 2000),
(6, 'Ushali','Saree','Clothing', 1, 1800),
(7, 'Deekshi','Oil','Grocery',5, 500),
(8, 'Deva','T-Shirt','Clothing', 2, 700),
```

```
(9, 'Muni','Pants','Clothing', 1, 1200),
(10,'Shruthi','Kurthiset','Clothing',2,2000);

--SELECT * FROM Orders;

SELECT OrderID,CustomerName,ProductName,Category,Quantity,Price, (Quantity
* Price) AS TotalAmount
FROM Orders;


SELECT Category, SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
ORDER BY TotalSales DESC;
LIMIT 1;

SELECT CustomerName, SUM(Quantity * Price) AS TotalSpent
FROM Orders
GROUP BY CustomerName
HAVING SUM(Quantity * Price) >
      (SELECT AVG(Quantity * Price) FROM Orders);


SELECT Category, SUM(Quantity * Price) AS TotalSales
FROM Orders
GROUP BY Category
HAVING SUM(Quantity * Price) > 5000;
```

Question-4

(Set Operators + Update/Delete + Complex Logic) A training institute

```
CREATE TABLE BatchA (
   StudentName VARCHAR(50) PRIMARY KEY
);

CREATE TABLE BatchB (
   StudentName VARCHAR(50) PRIMARY KEY
);

INSERT INTO BatchA VALUES
('Amulya'),
('Manju'),
('Ushali'),
('Deekshi'),
```

```
('Thulasi'),
('Praveen'),
('Ramya'),
('Jayalaxmi'),
('Deva'),
('Janu');
```

```
INSERT INTO BatchB VALUES
('Amulya'),
('Manju'),
('Meena'),
('Karthik'),
('Divya'),
('Rahul'),
('Rani'),
('Shruthi'),
('Amaani'),
('Amrin');
```

```
SELECT StudentName FROM BatchA
INTERSECT
SELECT StudentName FROM BatchB;
```

```
SELECT StudentName FROM BatchA
UNION
SELECT StudentName FROM BatchB;
```

```
SELECT StudentName FROM BatchA
EXCEPT
SELECT StudentName FROM BatchB;
```

```
CREATE TABLE Fees (
   StudentName VARCHAR(50),
   FeesPaid INT,
   CourseName VARCHAR(50)
);
```

```
INSERT INTO Fees VALUES
('Amulya', 5000, 'SQL'),
('Manju', 1500, 'SQL'),
('Ushali', 3500, 'Python'),
('Deekshi', 2500, 'SQL'),
('Thulasi', 1000, 'Java'),
('Praveen', 4000, 'SQL'),
('Rani', 3000, '.Net'),
('Shruthi',2500,'.Net'),
('Amaani',5000,'Python'),
('Amrin',4000,'Java');

UPDATE Fees
SET FeesPaid = FeesPaid + 1000
WHERE CourseName = 'SQL';

DELETE FROM Fees
WHERE FeesPaid < 2000;

SELECT
   StudentName,
   FeesPaid,
   CASE
      WHEN FeesPaid >= 3000 THEN 'Paid'
      ELSE 'Due'
   END AS Status
FROM Fees;
```

Question-5:
E-Commerce Order Integrity(Normalization + Constraints + Subquery)

```
--DROP TABLE IF EXISTS Customer;
CREATE TABLE Customer (
   CustomerID INT PRIMARY KEY,
   CustomerName VARCHAR(50) NOT NULL,
   CustomerEmail VARCHAR(50) UNIQUE NOT NULL
);
--DROP TABLE IF EXISTS Category;
CREATE TABLE Category (
   CategoryID INT PRIMARY KEY,
   CategoryName VARCHAR(50) NOT NULL
);
--DROP TABLE IF EXISTS Product;
CREATE TABLE Product (
```

```
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    CategoryID INT,
    Price DECIMAL(10,2) CHECK (Price >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

--DROP TABLE IF EXISTS Orderslist;
CREATE TABLE Orderslist (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
DROP TABLE IF EXISTS Detailsoforders;
CREATE TABLE Detailsoforders (
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT CHECK (Quantity > 0),
    FOREIGN KEY (OrderID) REFERENCES Orderslist(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);


INSERT INTO Customer VALUES
(1,'Amulya','ammu@gmail.com'),
(2,'Shruthi','sru@gmail.com'),
(3,'Amaani','Annu@gmail.com'),
(4,'Amrin','Ammu@gmail.com'),
(5,'Rahul','rahul@gmail.com');

INSERT INTO Category VALUES
(1,'Electronics'),
(2,'Clothing'),
(3,'Grocery');

INSERT INTO Product VALUES
(101,'Mobile',1,15000),
(102,'Laptop',1,55000),
(103,'Shirt',2,800),
(104,'Rice Bag',3,1200),
(105,'Headphones',1,2000);
```

```
INSERT INTO Orderslist VALUES
(201,1,'2026-02-10'),
(202,2,'2026-02-11'),
(203,3,'2026-02-12'),
(204,4,'2026-02-13'),
(205,5,'2026-02-14'),
(206,1,'2026-02-15'),
(207,2,'2026-02-16'),
(208,3,'2026-02-17');


INSERT INTO Detailsoforders VALUES
(1,201,101,1),
(2,202,102,1),
(3,203,103,2),
(4,204,104,3),
(5,205,105,2),
(6,206,103,1),
(7,207,104,2),
(8,208,101,1);


SELECT c.CustomerName, p.ProductName, ca.CategoryName, d.Quantity,
(d.Quantity * p.Price) AS TotalAmount
FROM Detailsoforders d
JOIN Orderslist o ON d.OrderID = o.OrderID
JOIN Customer c ON o.CustomerID = c.CustomerID
JOIN Product p ON d.ProductID = p.ProductID
JOIN Category ca ON p.CategoryID = ca.CategoryID;

SELECT c.CustomerName
FROM Orderslist o
JOIN Detailsoforders d ON o.OrderID = d.OrderID
JOIN Product p ON d.ProductID = p.ProductID
JOIN Customer c ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerName
HAVING COUNT(DISTINCT p.CategoryID) > 1;


SELECT p.ProductName, SUM(d.Quantity) AS TotalSold
FROM Detailsoforders d
JOIN Product p ON d.ProductID = p.ProductID
GROUP BY p.ProductName
ORDER BY TotalSold DESC
```

```
LIMIT 1;


DELETE FROM Product
WHERE ProductID NOT IN (
   SELECT ProductID FROM Detailsoforders
);
```

Question-6
Banking System(Transactions + Complex Logic)

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Phone VARCHAR(15) UNIQUE
);
--DROP TABLE IF EXISTS Accounts;
CREATE TABLE Accounts (
   AccountID INT PRIMARY KEY,
   UserID INT,
   Balance DECIMAL(12,2) CHECK (Balance >= 0),
   FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

CREATE TABLE Transactions (
   TransactionID INT PRIMARY KEY,
   AccountID INT,
   Type VARCHAR(10) CHECK (Type IN ('Credit','Debit')),
   Amount DECIMAL(10,2),
   Date DATE,
   FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);


INSERT INTO Users VALUES
(1,'Amulya','9000011111'),
(2,'Deva','9000022222'),
(3,'Ushali','9000033333'),
(4,'Janani','9000044444'),
(5,'Deekshi','9000055555');

INSERT INTO Accounts VALUES
(101,1,120000),
```

```
(102,1,5000),
(103,2,80000),
(104,3,150000),
(105,4,0),
(106,5,20000);


INSERT INTO Transactions(TransactionID,AccountID,type,Amount,Date)
VALUES
(1,101,'Credit',20000,'2026-02-01'),
(2,101,'Debit',5000,'2026-02-02'),
(3,102,'Credit',3000,'2026-02-03'),
(4,103,'Debit',10000,'2026-02-04'),
(5,103,'Credit',20000,'2026-02-05'),
(6,104,'Credit',50000,'2026-02-06'),
(7,104,'Debit',10000,'2026-02-07'),
(8,106,'Credit',7000,'2026-02-08'),
(9,106,'Debit',2000,'2026-02-09'),
(10,101,'Credit',15000,'2026-02-10');

SELECT u.Name AS CustomerName, a.AccountID,
SUM(CASE WHEN t.Type = 'Credit' THEN t.Amount ELSE 0 END) AS
TotalCredits,
SUM(CASE WHEN t.Type = 'Debit' THEN t.Amount ELSE 0 END) AS TotalDebits,
a.Balance AS CurrentBalance
FROM Users u
JOIN Accounts a ON u.UserID = a.UserID
LEFT JOIN Transactions t ON a.AccountID = t.AccountID
GROUP BY u.Name, a.AccountID, a.Balance;


--SELECT* FROM Accounts;


SELECT DISTINCT u.Name
FROM Users u
JOIN Accounts a ON u.UserID = a.UserID
JOIN Transactions t ON a.AccountID = t.AccountID
WHERE t.Amount >
    (SELECT AVG(t2.Amount)
     FROM Transactions t2
     WHERE t2.AccountID = t.AccountID);
```

```
UPDATE Accounts
SET Balance = Balance + (Balance * 0.02)
WHERE Balance > 100000;


DELETE FROM Accounts
WHERE Balance = 0 AND NOT EXISTS (
   SELECT 1
   FROM Transactions t
   WHERE t.AccountID = Accounts.AccountID
);
```

Question-7
University Grading System(CASE + Having + Ranking Logic)

```
--DROP TABLE IF EXISTS Departments;
CREATE TABLE Departments(
DeptID INT PRIMARY KEY,
DeptName VARCHAR(50) NOT NULL
);

--DROP TABLE IF EXISTS Students;
CREATE TABLE Students(
   StudentID INT PRIMARY KEY,
   Name VARCHAR(40),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

CREATE TABLE Marks (
   StudentID INT,
   Subject VARCHAR(30),
   Marks INT,
   FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

INSERT INTO Departments(DeptID,DeptName)
VALUES
(1,'CSE'),
(2,'ECE'),
(3,'AI&DS');

INSERT INTO Students(StudentID,Name,DeptID)
```

```
VALUES
(1,'Amulya',1),
(2,'Deva',1),
(3,'Ushali',2),
(4,'Janani',2),
(5,'Deekshi',3),
(6,'Ravi',3),
(7,'Anil',1),
(8,'Kiran',2);

INSERT INTO Marks VALUES
(1,'Math',90),(1,'DBMS',92),(1,'OS',88),
(2,'Math',70),(2,'DBMS',75),(2,'OS',80),
(3,'Math',60),(3,'DBMS',55),(3,'OS',65),
(4,'Math',85),(4,'DBMS',90),(4,'OS',87),
(5,'Math',40),(5,'DBMS',50),(5,'OS',45),
(6,'Math',78),(6,'DBMS',72),(6,'OS',75),
(7,'Math',95),(7,'DBMS',93),(7,'OS',91),
(8,'Math',68),(8,'DBMS',70),(8,'OS',72);


--select* From Departments;

SELECT s.StudentID, s.Name,
   SUM(m.Marks) AS TotalMarks
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name;

--overall grading
SELECT s.Name, SUM(m.Marks) AS TotalMarks,
   CASE
      WHEN SUM(m.Marks) >= 90 THEN 'A'
      WHEN SUM(m.Marks) >= 75 THEN 'B'
      WHEN SUM(m.Marks) >= 60 THEN 'C'
      ELSE 'Fail'
   END AS Grade
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name;


--subjectwise grading
SELECT s.Name, m.Subject, m.Marks,
```

```
    CASE
        WHEN m.Marks >= 90 THEN 'A'
        WHEN m.Marks >= 75 THEN 'B'
        WHEN m.Marks >= 60 THEN 'C'
        ELSE 'Fail'
    END AS Grade
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID;


SELECT d.DeptName, AVG(m.Marks) AS AvgMarks
FROM Departments d
JOIN Students s ON d.DeptID = s.DeptID
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY d.DeptName
ORDER BY AvgMarks DESC
LIMIT 1;


SELECT s.Name, s.DeptID, AVG(m.Marks) AS StudentAvg
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name, s.DeptID
HAVING AVG(m.Marks) >
      (SELECT AVG(m2.Marks)
       FROM Students s2
       JOIN Marks m2 ON s2.StudentID = m2.StudentID
       WHERE s2.DeptID = s.DeptID);


--GROUP BY/HAVING
SELECT d.DeptName, AVG(m.Marks) AS AvgMarks
FROM Departments d
JOIN Students s ON d.DeptID = s.DeptID
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY d.DeptName
HAVING AVG(m.Marks) > 70;
```

Question-8

Inventory + Supplier System(Advanced Join + Set Logic

```
CREATE TABLE Suppliers (
   SupplierID INT PRIMARY KEY,
   Name VARCHAR(50)
);

CREATE TABLE Products (
   ProductID INT PRIMARY KEY,
   Name VARCHAR(50),
   SupplierID INT,
   StockQty INT,
   FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID)
);

CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   ProductID INT,
   QuantityOrdered INT,
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

INSERT INTO Suppliers VALUES
(1,'Amulya Traders'),
(2,'Deva Suppliers'),
(3,'Manju Distributors'),
(4,'Local Store'),
(5,'Global Supply');

--SELECT * FROM Orders;

INSERT INTO Products VALUES
(101,'Laptop',1,300),
(102,'Mouse',1,50),
(103,'Keyboard',2,150),
(104,'Monitor',2,600),
(105,'Rice Bag',3,80),
(106,'Sugar Pack',3,40);

INSERT INTO Orders VALUES
(1,101,200),
(2,101,150),
```

```
(3,102,100),
(4,103,50),
(5,104,100),
(6,105,120),
(7,105,50),
(8,106,70);

--Suppliers more than 1
SELECT SupplierID, COUNT(ProductID) AS TotalProducts
FROM Products
GROUP BY SupplierID
HAVING COUNT(ProductID) > 1;
--stock less than order quantity
SELECT p.Name, p.StockQty, SUM(o.QuantityOrdered) AS OrderedQty
FROM Products p
JOIN Orders o ON p.ProductID = o.ProductID
GROUP BY p.ProductID
HAVING p.StockQty < SUM(o.QuantityOrdered);


SELECT s.Name
FROM Suppliers s
LEFT JOIN Products p ON s.SupplierID = p.SupplierID
WHERE p.ProductID IS NULL;


UPDATE Products
SET StockQty = StockQty + 50
WHERE StockQty < 100;


DELETE FROM Suppliers
WHERE SupplierID NOT IN (
   SELECT SupplierID FROM Products
);


SELECT Name, StockQty,
   CASE
      WHEN StockQty >= 500 THEN 'High Stock'
      WHEN StockQty >= 200 THEN 'Medium Stock'
      ELSE 'Low Stock'
   END AS StockStatus
FROM Products;
```

Question-9
Movie Streaming Platform(Tricky Analytics)

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY,
   Name VARCHAR(50),
   SubscriptionType VARCHAR(20)
);

CREATE TABLE Movies (
   MovieID INT PRIMARY KEY,
   Title VARCHAR(50),
   Genre VARCHAR(30)
);

CREATE TABLE WatchHistory (
   UserID INT,
   MovieID INT,
   WatchTime INT,
   FOREIGN KEY (UserID) REFERENCES Users(UserID),
   FOREIGN KEY (MovieID) REFERENCES Movies(MovieID)
);

INSERT INTO Users VALUES
(1,'Amulya','Basic'),
(2,'Shruthi','Basic'),
(3,'Amaani','Premium'),
(4,'Amrin','Basic'),
(5,'Mamatha','Basic');

INSERT INTO Movies VALUES
(101,'Avatar','Sci-Fi'),
(102,'Titanic','Romance'),
(103,'Avengers','Action'),
(104,'Bahubali','Action'),
(105,'Lion King','Animation');

INSERT INTO WatchHistory VALUES
(1,101,120),
(1,102,90),
(1,103,150),
(2,103,100),
(2,104,140),
(3,101,200),
```

```
(3,105,80),
(4,102,60),
(4,105,50);

SELECT m.Title, SUM(w.WatchTime) AS TotalWatch
FROM Movies m
JOIN WatchHistory w ON m.MovieID = w.MovieID
GROUP BY m.MovieID
ORDER BY TotalWatch DESC
LIMIT 1;

SELECT u.Name, COUNT(DISTINCT m.Genre) AS GenreCount
FROM Users u
JOIN WatchHistory w ON u.UserID = w.UserID
JOIN Movies m ON w.MovieID = m.MovieID
GROUP BY u.UserID
HAVING COUNT(DISTINCT m.Genre) > 2;

SELECT m.Genre, SUM(w.WatchTime) AS TotalTime
FROM Movies m
JOIN WatchHistory w ON m.MovieID = w.MovieID
GROUP BY m.Genre;

UPDATE Users
SET SubscriptionType = 'Premium'
WHERE UserID IN (
   SELECT UserID
   FROM WatchHistory
   GROUP BY UserID
   HAVING SUM(WatchTime) >
         (SELECT AVG(TotalTime)
          FROM (SELECT SUM(WatchTime) AS TotalTime
               FROM WatchHistory
               GROUP BY UserID))
);


SELECT Title
FROM Movies m
WHERE NOT EXISTS (
   SELECT 1
   FROM WatchHistory w
   WHERE w.MovieID = m.MovieID
);
```

Question-10
HR Promotion Logic (Very Interview-Oriented)

```
--DROP TABLE IF EXISTS Departments;
CREATE TABLE Departments (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);
--DROP TABLE IF EXISTS Employees;
CREATE TABLE Employees (
   EmpID INT PRIMARY KEY,
   Name VARCHAR(50),
   Salary DECIMAL(10,2),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
--DROP TABLE IF EXISTS Performance;
CREATE TABLE Performance (
   EmpID INT,
   Rating INT,
   FOREIGN KEY (EmpID) REFERENCES Employees(EmpID)
);


INSERT INTO Departments VALUES
(1,'IT'),
(2,'HR'),
(3,'Finance');

INSERT INTO Employees VALUES
(101,'Amulya',50000,1),
(102,'Ravi',60000,1),
(103,'Anu',45000,2),
(104,'Kiran',70000,3),
(105,'Meena',30000,2),
(106,'Arun',80000,3);

INSERT INTO Performance VALUES
(101,4),
(102,5),
(103,3),
(104,4),
(105,1),
(106,5);
```

```
--increase salry by 10%
UPDATE Employees
SET Salary = Salary + (Salary * 0.10)
WHERE EmpID IN (
   SELECT EmpID FROM Performance WHERE Rating >= 4
);
--Perfomance level rating
SELECT e.Name, d.DeptName, e.Salary, p.Rating,
   CASE
      WHEN p.Rating = 5 THEN 'Excellent'
      WHEN p.Rating = 4 THEN 'Good'
      WHEN p.Rating = 3 THEN 'Average'
      ELSE 'Poor'
  END AS PerformanceLevel
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
JOIN Performance p ON e.EmpID = p.EmpID;

--Department with highest salary
SELECT d.DeptName, AVG(e.Salary) AS AvgSalary
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
GROUP BY d.DeptID
ORDER BY AvgSalary DESC
LIMIT 1;

--employees earning above department average.
SELECT Name, Salary, DeptID
FROM Employees e
WHERE Salary > (
   SELECT AVG(Salary)
   FROM Employees
   WHERE DeptID = e.DeptID
);
--deleting emp <2 rating.
DELETE FROM Employees
WHERE EmpID IN (
   SELECT EmpID FROM Performance WHERE Rating < 2
);
```

Question-11
Fraud Detection Logic

```
CREATE TABLE Transactions (
   TransactionID INT PRIMARY KEY,
   UserID INT,
   Amount DECIMAL(10,2),
   Location VARCHAR(50),
   TransDate DATE
);

INSERT INTO Transactions VALUES
(1,101,5000,'Bangalore','2026-02-20'),
(2,101,120000,'Bangalore','2026-02-20'),
(3,101,7000,'Bangalore','2026-02-20'),
(4,101,3000,'Bangalore','2026-02-20'),
(5,102,1500,'Hyderabad','2026-02-21'),
(6,102,2500,'Chennai','2026-02-21'),
(7,103,90000,'Chennai','2026-02-22'),
(8,103,110000,'Chennai','2026-02-22'),
(9,104,2000,'Hyderabad','2018-01-10');

--SELECT * FROM Transactions;
SELECT UserID, TransDate, COUNT(TransactionID) AS TotalTrans
FROM Transactions
GROUP BY UserID, TransDate
HAVING COUNT(TransactionID) > 3;

--user transaction  more than 1 location
SELECT UserID, COUNT(DISTINCT Location) AS LocationCount
FROM Transactions
GROUP BY UserID
HAVING COUNT(DISTINCT Location) > 1;

--suspecious recards
SELECT TransactionID, UserID, Amount, Location,
   CASE
      WHEN Amount > 100000 THEN 'High Risk'
      WHEN UserID IN (
       SELECT UserID FROM Transactions
       GROUP BY UserID
       HAVING COUNT(DISTINCT Location) > 1
       ) THEN 'Location Risk'
       ELSE 'Normal'
```

```
    END AS RiskStatus
FROM Transactions;

--older than 5 yeras trans
DELETE FROM Transactions
WHERE TransDate < DATE('now','-5 years');
```

25/02/2026 **SQL PROJECTS**

PROJECT 1 — Smart Hospital Management & Billing Analytics System

```
CREATE TABLE Patients (
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(50) NOT NULL,
    Phone VARCHAR(15) UNIQUE,
    Email VARCHAR(50) UNIQUE
);

CREATE TABLE Specializations (
    SpecID INT PRIMARY KEY,
    SpecName VARCHAR(50)
);
CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(50),
    SpecID INT,
    FOREIGN KEY (SpecID) REFERENCES Specializations(SpecID)
);
CREATE TABLE Appointments (
    ApptID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    ApptDate DATE NOT NULL,
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);
CREATE TABLE Treatments (
    TreatmentID INT PRIMARY KEY,
    TreatmentName VARCHAR(50),
    Cost DECIMAL(10,2)
);
CREATE TABLE InsuranceProviders (
```

```
    InsuranceID INT PRIMARY KEY,
    ProviderName VARCHAR(50)
);

-- Bills table
CREATE TABLE Bills (
    BillID INT PRIMARY KEY,
    ApptID INT,
    TreatmentID INT,
    InsuranceID INT,
    BillAmount DECIMAL(10,2) CHECK (BillAmount > 0),
    InsuranceCover DECIMAL(10,2),
    FOREIGN KEY (ApptID) REFERENCES Appointments(ApptID),
    FOREIGN KEY (TreatmentID) REFERENCES Treatments(TreatmentID),
    FOREIGN KEY (InsuranceID) REFERENCES InsuranceProviders(InsuranceID)
);


-- Patients
INSERT INTO Patients (PatientID,PatientName,Phone,Email)
VALUES
(1,'Ramamurthy','9001111111','ram@gmail.com'),
(2,'Ravi','9002222222','ravi@gmail.com'),
(3,'Anu','9003333333','anu@gmail.com'),
(4,'Kiran','9004444444','kiran@gmail.com');

--SELECT * FROM Doctors;
-- Specializations
INSERT INTO Specializations(SpecID ,SpecName)
VALUES
(1,'Cardiology'),
(2,'Neurology'),
(3,'Orthopedics');
--SELECT * FROM Specializations;

-- Doctors
INSERT INTO Doctors(DoctorID,DoctorName,SpecID)
VALUES
(101,'Dr Dhoni',1),
(102,'Dr Virat',2),
(103,'Dr Rohit',3);
SELECT * FROM Doctors;

-- Appointments
```

```
INSERT INTO Appointments(ApptID,PatientID,DoctorID,ApptDate)
VALUES
(1001,1,101,'2026-02-20'),
(1002,2,102,'2026-02-21'),
(1003,3,103,'2026-02-22'),
(1004,1,102,'2023-01-10');
--SELECT * FROM Appointments;

-- Treatments
INSERT INTO Treatments(TreatmentID, TreatmentName, Cost)
VALUES
(201,'Heart Checkup',5000),
(202,'Brain Scan',8000),
(203,'Bone Surgery',15000);
--SELECT * FROM Treatments;

-- Insurance
INSERT INTO InsuranceProviders(InsuranceID,ProviderName)
VALUES
(301,'Star Health'),
(302,'LIC Health');

--SELECT * FROM InsuranceProviders;
-- Bills
INSERT INTO Bills VALUES
(401,1001,201,301,5000,2000),
(402,1002,202,302,8000,3000),
(403,1003,203,301,15000,5000),
(404,1004,201,NULL,5000,0);

--SELECT * FROM Bills;


--patient billing report
SELECT
p.PatientName,d.DoctorName,s.SpecName,t.TreatmentName,a.ApptDate,b.BillAmou
nt,b.InsuranceCover,
      (b.BillAmount - b.InsuranceCover) AS FinalPayableAmount
FROM Bills b
JOIN Appointments a ON b.ApptID = a.ApptID
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecID = s.SpecID
JOIN Treatments t ON b.TreatmentID = t.TreatmentID;
```

```
--top 3 revenue generating doctors
SELECT d.DoctorName, SUM(b.BillAmount) AS TotalRevenue
FROM Doctors d
JOIN Appointments a ON d.DoctorID = a.DoctorID
JOIN Bills b ON a.ApptID = b.ApptID
GROUP BY d.DoctorID
ORDER BY TotalRevenue DESC
LIMIT 3;

--Patient payment Status
SELECT p.PatientName, (b.BillAmount - b.InsuranceCover) AS FinalPayable,
   CASE
      WHEN (b.BillAmount - b.InsuranceCover) > 0 THEN 'Pending'
      ELSE 'Paid'
   END AS PaymentStatus
FROM Bills b
JOIN Appointments a ON b.ApptID = a.ApptID
JOIN Patients p ON a.PatientID = p.PatientID;

--Specialization above average revenue
SELECT s.SpecName, SUM(b.BillAmount) AS TotalRevenue
FROM Specializations s
JOIN Doctors d ON s.SpecID = d.SpecID
JOIN Appointments a ON d.DoctorID = a.DoctorID
JOIN Bills b ON a.ApptID = b.ApptID
GROUP BY s.SpecID
HAVING SUM(b.BillAmount) >
      (SELECT AVG(TotalRev)
      FROM (
         SELECT SUM(b.BillAmount) AS TotalRev
         FROM Specializations s
         JOIN Doctors d ON s.SpecID = d.SpecID
         JOIN Appointments a ON d.DoctorID = a.DoctorID
         JOIN Bills b ON a.ApptID = b.ApptID
         GROUP BY s.SpecID
      ));

--Deleting inactive patients list
DELETE FROM Patients
WHERE NOT EXISTS (
   SELECT 1 FROM Appointments a
   WHERE a.PatientID = Patients.PatientID
     AND a.ApptDate >= DATE('now','-3 years')
```

```
);

--Bonus for high revenue doctors
UPDATE Doctors
SET DoctorName = DoctorName || ' (Bonus)'
WHERE DoctorID IN (
   SELECT d.DoctorID
   FROM Doctors d
   JOIN Appointments a ON d.DoctorID = a.DoctorID
   JOIN Bills b ON a.ApptID = b.ApptID
   GROUP BY d.DoctorID
   HAVING SUM(b.BillAmount) > 1000000
);
```

PROJECT 2 — Online Marketplace with Seller Performance & Fraud Detection

```
-- USERS TABLE (buyers + sellers)
CREATE TABLE Users (
   UserID INTEGER PRIMARY KEY,
   Name TEXT NOT NULL,
   Email TEXT UNIQUE,
   UserType TEXT CHECK(UserType IN ('Buyer','Seller'))
);

-- CATEGORIES
CREATE TABLE Categories (
   CategoryID INTEGER PRIMARY KEY,
   CategoryName TEXT
);

-- PRODUCTS
--DROP TABLE IF EXISTS Products;
CREATE TABLE Products (
   ProductID INTEGER PRIMARY KEY,
   ProductName TEXT,
   Price REAL CHECK(Price > 0),
   Quantity INTEGER CHECK(Quantity > 0),
   SellerID INTEGER,
   CategoryID INTEGER,
   FOREIGN KEY (SellerID) REFERENCES Users(UserID),
   FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

-- ORDERS
```

```
CREATE TABLE Orders (
   OrderID INTEGER PRIMARY KEY,
   BuyerID INTEGER,
   OrderDate DATE,
   FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);

-- ORDER DETAILS
CREATE TABLE OrderDetails (
   OrderDetailID INTEGER PRIMARY KEY,
   OrderID INTEGER,
   ProductID INTEGER,
   Qty INTEGER,
   FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE
CASCADE,
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- PAYMENTS
CREATE TABLE Payments (
   PaymentID INTEGER PRIMARY KEY,
   OrderID INTEGER,
   Amount REAL,
   RefundFlag INTEGER DEFAULT 0, -- 1 = refunded
   FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);

-- REVIEWS
CREATE TABLE Reviews (
   ReviewID INTEGER PRIMARY KEY,
   ProductID INTEGER,
   Rating INTEGER CHECK(Rating BETWEEN 1 AND 5),
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);


INSERT INTO Users VALUES
(1, 'Amulya','amulya@gmail.com','Seller'),
(2, 'Amaani','amaani@gmail.com','Seller'),
(3, 'shruthi','shruthi@gmail.com','Seller'),
(4, 'Amrin','amrin@gmail.com','Buyer'),
(5, 'mamatha','mama@gmail.com','Buyer'),
(6, 'Dharani','dharani@gmail.com','Buyer');
```

```
--SELECT* FROM Users;

INSERT INTO Categories VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Books');

--SELECT* FROM Categories;

INSERT INTO Products VALUES
(1, 'Mobile Phone', 15000, 50, 1, 1),
(2, 'Laptop', 55000, 30, 1, 1),
(3, 'T-Shirt', 800, 100, 2, 2),
(4, 'Jeans', 1200, 80, 2, 2),
(5, 'SQL Book', 500, 200, 3, 3);

--SELECT* FROM Products;

INSERT INTO Orders VALUES
(1, 4, '2025-02-20'),
(2, 5, '2025-02-20'),
(3, 6, '2025-02-21'),
(4, 4, '2025-02-21'),
(5, 4, '2025-02-21'),
(6, 4, '2025-02-21');

--SELECT* FROM Orders;

INSERT INTO OrderDetails VALUES
(1, 1, 1, 1),
(2, 1, 3, 2),
(3, 2, 2, 1),
(4, 3, 5, 3),
(5, 4, 4, 1),
(6, 5, 3, 5),
(7, 6, 1, 2);

--SELECT* FROM OrderDetails;

INSERT INTO Payments VALUES
(1, 1, 15000, 0),
(2, 2, 55000, 1),
(3, 3, 1500, 0),
(4, 4, 1200, 0),
```

```
(5, 5, 4000, 1),
(6, 6, 30000, 0);

--SELECT* FROM Payments;

INSERT INTO Reviews VALUES
(1, 1, 5),
(2, 2, 4),
(3, 3, 3),
(4, 5, 5);

--SELECT* FROM Reviews;


--seller performance dashboard
SELECT
   u.Name AS SellerName,
   COUNT(DISTINCT p.ProductID) AS TotalProducts,
   COUNT(DISTINCT o.OrderID) AS TotalOrders,
   SUM(pay.Amount) AS TotalRevenue,
   AVG(r.Rating) AS AverageRating,

   CASE
      WHEN SUM(pay.Amount) > 500000 THEN 'Platinum'
      WHEN SUM(pay.Amount) >= 200000 THEN 'Gold'
      ELSE 'Silver'
   END AS PerformanceLevel

FROM Users u
LEFT JOIN Products p ON u.UserID = p.SellerID
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
LEFT JOIN Orders o ON od.OrderID = o.OrderID
LEFT JOIN Payments pay ON o.OrderID = pay.OrderID
LEFT JOIN Reviews r ON p.ProductID = r.ProductID
WHERE u.UserType = 'Seller'
GROUP BY u.UserID;


--Buyers with more than 5 orders.
SELECT BuyerID, OrderDate, COUNT(*) AS TotalOrders
FROM Orders
GROUP BY BuyerID, OrderDate
HAVING COUNT(*) > 5;
```

```sql
--Buyers with refunds > 50%
SELECT o.BuyerID
FROM Orders o
JOIN Payments p ON o.OrderID = p.OrderID
GROUP BY o.BuyerID
HAVING SUM(p.RefundFlag) * 1.0 / COUNT(*) > 0.5;

--MOST PROFITABLE CATEGORY
SELECT c.CategoryName, SUM(pay.Amount) AS Revenue
FROM Categories c
JOIN Products p ON c.CategoryID = p.CategoryID
JOIN OrderDetails od ON p.ProductID = od.ProductID
JOIN Payments pay ON od.OrderID = pay.OrderID
GROUP BY c.CategoryID
ORDER BY Revenue DESC
LIMIT 1;

--Sellers with no products OR no orders in last 1 year
DELETE FROM Users
WHERE UserType = 'Seller'
AND (
   UserID NOT IN (SELECT SellerID FROM Products)
   OR
   UserID NOT IN (
      SELECT p.SellerID
      FROM Products p
      JOIN OrderDetails od ON p.ProductID = od.ProductID
      JOIN Orders o ON od.OrderID = o.OrderID
      WHERE o.OrderDate >= DATE('now','-1 year')
   )
);
--Apply 10% discount to slow moving products
UPDATE Products
SET Price = Price * 0.9
WHERE ProductID IN (
   SELECT p.ProductID
   FROM Products p
   JOIN OrderDetails od ON p.ProductID = od.ProductID
   GROUP BY p.ProductID
   HAVING SUM(od.Qty) < (
      SELECT AVG(Qty)
      FROM OrderDetails
   )
);
```

```
--products never reviewed
SELECT ProductName
FROM Products p
WHERE NOT EXISTS (
   SELECT 1 FROM Reviews r WHERE r.ProductID = p.ProductID
);
```

PROJECT 3— "Global Ride-Sharing Analytics & Compliance System"

```
CREATE TABLE Cities(
   CityID INTEGER PRIMARY KEY,
   CityName TEXT UNIQUE
);

CREATE TABLE Drivers(
   DriverID INTEGER PRIMARY KEY,
   Name TEXT,
   Phone TEXT UNIQUE,
   CityID INTEGER,
   JoinDate DATE,
   FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);

CREATE TABLE Riders(
   RiderID INTEGER PRIMARY KEY,
   Name TEXT,
   Phone TEXT UNIQUE
);

CREATE TABLE Vehicles(
   VehicleID INTEGER PRIMARY KEY,
   DriverID INTEGER,
   Type TEXT,
   PlateNumber TEXT UNIQUE,
   FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
);

CREATE TABLE Trips(
   TripID INTEGER PRIMARY KEY,
   DriverID INTEGER,
```

```
    RiderID INTEGER,
    CityID INTEGER,
    TripDate DATE,
    DistanceKM REAL CHECK(DistanceKM > 0),
    FareAmount REAL CHECK(FareAmount > 0),
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID),
    FOREIGN KEY (RiderID) REFERENCES Riders(RiderID),
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);
DROP TABLE IF EXISTS Payments;
CREATE TABLE Payments(
    PaymentID INTEGER PRIMARY KEY,
    TripID INTEGER,
    PaymentMethod TEXT,
    PaidAmount REAL,
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);

CREATE TABLE DriverRatings(
    RatingID INTEGER PRIMARY KEY,
    TripID INTEGER,
    RatingScore INTEGER CHECK(RatingScore BETWEEN 1 AND 5),
    FOREIGN KEY (TripID) REFERENCES Trips(TripID) ON DELETE CASCADE
);



-- Cities
INSERT INTO Cities VALUES
(1,'Bangalore'),
(2,'Hyderabad'),
(3,'Chennai');

-- Drivers
INSERT INTO Drivers VALUES
(1,'Ravi','9001110001',1,'2022-01-01'),
(2,'Kiran','9001110002',1,'2023-01-01'),
(3,'Amit','9001110003',2,'2022-05-01'),
(4,'John','9001110004',2,'2021-03-01'),
(5,'Suresh','9001110005',3,'2020-02-01');

-- Riders
INSERT INTO Riders VALUES
(1,'Jhansi','8001110001'),
```

```
(2,'Rahul','8001110002'),
(3,'Neha','8001110003');

-- Vehicles
INSERT INTO Vehicles VALUES
(1,1,'Car','KA01AB1234'),
(2,2,'Bike','KA02AB1234'),
(3,3,'Car','TS01AB1234'),
(4,4,'Car','TS02AB1234'),
(5,5,'Auto','TN01AB1234');

-- Trips
INSERT INTO Trips VALUES
(1,1,1,1,'2026-02-01',10,500),
(2,1,2,1,'2026-02-01',5,300),
(3,2,3,1,'2026-02-02',2,150),
(4,3,1,2,'2026-02-01',20,2000),
(5,3,2,2,'2026-02-02',25,2500),
(6,4,3,2,'2026-02-01',1,8000),
(7,5,1,3,'2026-02-01',15,1000);

-- Payments
INSERT INTO Payments VALUES
(1,1,'Card',500),
(2,2,'Cash',300),
(3,3,'Card',150),
(4,4,'Card',2000),
(5,5,'Cash',2500),
(6,6,'Cash',8000),
(7,7,'Card',1000);

-- Ratings
INSERT INTO DriverRatings VALUES
(1,1,5),
(2,2,4),
(3,3,3),
(4,4,4),
(5,5,2),
(6,6,1),
(7,7,5);

--underperforming drivers
SELECT d.Name, c.CityName,
       AVG(r.RatingScore) AS AvgRating,
```

```
      COUNT(t.TripID) AS TotalTrips,
      CASE
        WHEN AVG(r.RatingScore) < (
            SELECT AVG(r2.RatingScore)
            FROM Trips t2 JOIN DriverRatings r2 ON t2.TripID=r2.TripID
            WHERE t2.CityID = d.CityID
        )
        AND COUNT(t.TripID) < (
            SELECT AVG(cnt)
            FROM (SELECT COUNT(*) cnt FROM Trips WHERE CityID=d.CityID
GROUP BY DriverID)
        )
        THEN 'Underperforming'
        WHEN AVG(r.RatingScore) >= 4 THEN 'Excellent'
        ELSE 'Stable'
      END AS Status
FROM Drivers d
JOIN Cities c ON d.CityID=c.CityID
LEFT JOIN Trips t ON d.DriverID=t.DriverID
LEFT JOIN DriverRatings r ON t.TripID=r.TripID
GROUP BY d.DriverID;

--revenue consistency
SELECT DriverID
FROM Trips
GROUP BY DriverID
HAVING COUNT(*) >= 5
AND MAX(FareAmount) > 3 * AVG(FareAmount);

--City Revenue
SELECT CityID
FROM Trips
GROUP BY CityID
HAVING SUM(FareAmount) * 0.6 <
(
   SELECT SUM(FareAmount)
   FROM Trips t2
   WHERE t2.CityID = Trips.CityID
);

--riders using multiple cities same day
SELECT RiderID, TripDate
FROM Trips
GROUP BY RiderID, TripDate
```

```
HAVING COUNT(DISTINCT CityID) > 1;

--driver loyalty bonus update
UPDATE Trips
SET FareAmount = FareAmount * 1.05
WHERE DriverID IN (
   SELECT DriverID
   FROM Trips
   GROUP BY DriverID
   HAVING COUNT(*) > (
      SELECT AVG(cnt)
      FROM (SELECT COUNT(*) cnt FROM Trips GROUP BY CityID)
   )
)
AND DriverID IN (
   SELECT t.DriverID
   FROM Trips t JOIN DriverRatings r ON t.TripID=r.TripID
   GROUP BY t.DriverID
   HAVING AVG(r.RatingScore) > (SELECT AVG(RatingScore) FROM
DriverRatings)
);


--delete inactive drivers
DELETE FROM Drivers d
WHERE NOT EXISTS (
   SELECT 1 FROM Trips t WHERE t.DriverID=d.DriverID
)
OR NOT EXISTS (
   SELECT 1 FROM Trips t
   WHERE t.DriverID=d.DriverID AND t.TripDate >= date('now','-2 years')
);

--Frouddetectionlabel
SELECT t.TripID, d.Name, c.CityName,
CASE
 WHEN t.DistanceKM < 1 AND t.FareAmount > (
     SELECT AVG(FareAmount) FROM Trips WHERE CityID=t.CityID
 ) THEN 'Suspicious'
 WHEN p.PaymentMethod='Cash' AND t.FareAmount>5000 THEN 'High Risk'
 ELSE 'Normal'
END AS RiskLevel
FROM Trips t
JOIN Drivers d ON t.DriverID=d.DriverID
```

```
JOIN Cities c ON t.CityID=c.CityID
JOIN Payments p ON t.TripID=p.TripID;

--multi level business metric
SELECT CityID
FROM Trips
GROUP BY CityID
HAVING AVG(FareAmount/DistanceKM) > (
   SELECT AVG(FareAmount/DistanceKM) FROM Trips
)
AND COUNT(*) > (
   SELECT AVG(cnt) FROM (SELECT COUNT(*) cnt FROM Trips GROUP BY
CityID)
);

--set operator
-- A = High rating drivers
SELECT DriverID FROM Trips t
JOIN DriverRatings r ON t.TripID=r.TripID
GROUP BY DriverID
HAVING AVG(r.RatingScore)>=4

INTERSECT

-- B = High revenue drivers
SELECT DriverID FROM Trips
GROUP BY DriverID
HAVING SUM(FareAmount) > (
   SELECT AVG(SUM(FareAmount)) FROM Trips GROUP BY CityID
);
--integrity audit query
SELECT t.TripID, t.FareAmount, p.PaidAmount, d.CityID, t.CityID
FROM Trips t
JOIN Payments p ON t.TripID=p.TripID
JOIN Drivers d ON t.DriverID=d.DriverID
WHERE p.PaidAmount != t.FareAmount
OR d.CityID != t.CityID;
```

26/02/2026 HTML , CSS, JAVASCRIPT PROJECTS

Project 1 — "Smart College Event Registration Portal"

File-1:Home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart College Event Registration Portal</title>
    <link rel="stylesheet" href="collegeevent.css">
</head>
<body>
    <header>
        <h1>Smart College Event Registration Portal</h1>
    </header>

    <nav>
        <a href="home.html">Home</a>
        <a href="events.html">Events</a>
        <a href="register.html">Register</a>
    </nav>

    <section>
        <h2>Welcome Student!</h2>
        <p id="WelcomeMSG"></p>
        <article>
            <h3>Upcoming Event Schedule</h3>
            <ol>
            <li>Tech Symposium</li>
            <li>Cultural Fest</li>
            <li>Sports Meet</li>
        </ol>
        </article>
    </section>
    <footer>
        <p>© 2026 Smart College</p>
    </footer>
    <script src="script.js"></script>

</body>
</html>
```

File-2:event.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Event</title>
   <link rel="stylesheet" href="collegeevent.css">
</head>
<body>
   <header>
   <h1>Events</h1>
</header>
<nav>
   <a href="home.html">Home</a> >
   <a href="events.html">Events</a>
</nav>
<section>
   <h2>Event Details</h2>
   <table>
      <tr>
         <th>Event</th>
         <th>Date</th>
         <th>Venue</th>
         <th>Seats Left</th>
      </tr>
      <tbody id="eventTable"></tbody>
   </table>
   <h3>Rules</h3>
   <ul>
      <li>Carry ID card</li>
      <li>Maintain discipline</li>
      <li>Report 30 mins early</li>
   </ul>
</section>
<footer>
   <p>All events are subject to change.</p>
</footer>
<script src="script.js"></script>
</body>
```

```
</html>
```

File-3:register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>register</title>
   <link rel="stylesheet" href="collegeevent.css">
</head>
<body>
   <header>
   <h1>Event Registration</h1>
</header>
<nav>
   <a href="home.html">Home</a> >
   <a href="events.html">Events</a> >
   <a href="register.html">Register</a>
</nav>
<section>
   <form id="refForm">
      Name:
      <input type="text" id="name" required><br><br>

      Email:
      <input type="email" id="email" required><br><br>

       Age:
       <input type="number" id="age" min="17" max="30" required><br><br>

       Event Date:
       <input type="date" required><br><br>

       Department:
       <input list="deptList" id="dept" required>
       <datalist id="deptList">
         <option value="CSE">
         <option value="ECE">
         <option value="EEE">
         <option value="Mechanical">
```

```
        </datalist><br><br>

        Gender:
        <input type="radio" name="Gender" value="Male">Male
        <input type="radio" name="Gender" value="Female">Female<br><br>

        Events Interested:
        <input type="checkbox" value="Tech"> Tech
        <input type="checkbox" value="Cultural"> Cultural
        <input type="checkbox" value="Sports"> Sports<br><br>

        Form Completion:
        <progress id="formProgress" value="0" max="100"></progress><br><br>

        Seat Availability:
        <meter value="70" min="0" max="100"></meter><br><br>

        <button type="button" onclick="registerStudent()">Submit</button>
 </form>
 <h3 id="confirmMsg"></h3>

 <button onclick="getLocation()">Get My Location</button>
<p id="location"></p>
</section>
<script src="script.js"></script>

</body>
</html>
```

File-4:collegeevent.css

```
body {
    font-family: Arial;
    margin: 0;
    padding: 0;
    background-color: #f4f6f8;
}

header {
    background: #2c3e50;
    color: white;
    padding: 15px;
    text-align: center;
}
```

```
nav {
    background:steelblue;
    padding: 10px;
    text-align: center;
}

nav a {
    color: white;
    text-decoration: none;
    margin: 10px;
}

section {
    padding: 20px;
}

form {
    background-color: rgb(0, 255, 255);
    background:skyblue;
    padding: 20px;
    border: 1px solid #ccc;
    width: 60%;
    margin: auto;
}

/* Flexbox layout */
section {
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* Responsive */
@media (max-width: 768px) {
    form {
        width: 90%;
    }
}
```

File-5:script.js

```
// Event Data
let events = [
    { name: "Tech Symposium", date: "2026-03-10", venue: "Auditorium", seats: 70 },
    { name: "Cultural Fest", date: "2026-03-15", venue: "Open Ground", seats: 50 },
    { name: "Sports Meet", date: "2026-03-20", venue: "Sports Complex", seats: 40 }
];

// Load events into table
if (document.getElementById("eventTable")) {
    let table = document.getElementById("eventTable");
    for (let i = 0; i < events.length; i++) {
        table.innerHTML += `
            <tr>
                <td>${events[i].name}</td>
                <td>${events[i].date}</td>
                <td>${events[i].venue}</td>
                <td>${events[i].seats}</td>
            </tr>
        `;
    }
}

// Register Function
function registerStudent() {

    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    let age = document.getElementById("age").value;

    if (name === "" || email === "" || age === "") {
        alert("Please fill all required fields");
        return;
    }

    if (age < 17) {
        alert("You must be at least 17 years old");
    } else {
        localStorage.setItem("studentName", name);
        document.getElementById("confirmMsg").innerText =
            "Registration successful! Welcome " + name;
    }
}

// Show stored name on home page
```

```
window.onload = function () {
    let storedName = localStorage.getItem("studentName");
    if (storedName && document.getElementById("welcomeMsg")) {
        document.getElementById("welcomeMsg").innerText =
            "Welcome back, " + storedName;
    }
};

// Progress update
document.addEventListener("change", function () {
    let progress = document.getElementById("formProgress");
    if (progress) {
        progress.value += 20;
    }
});

// Geolocation
function getLocation() {
    if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(
            function (position) {
                document.getElementById("location").innerText =
                    "Latitude: " + position.coords.latitude +
                    " , Longitude: " + position.coords.longitude;
            },
            function () {
                document.getElementById("location").innerText =
                    "Permission denied or location unavailable.";
            }
        );
    } else {
        alert("Geolocation not supported");
    }
}
```

Project 2 — "Personal Finance Tracker (Client-Side Only)"

File-1:finance.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My Finance Tracker</title>
```

```html
    <link rel="stylesheet" href="finance.css">
</head>
<body>

<header>
    <h1>Personal Finance Tracker</h1>
</header>

<section class="layout">

    <article class="form-box">
        <h2>Add Transaction</h2>

        <form id="txnForm">
            <input type="text" id="title" placeholder="Description" required>

            <input type="number" id="amount" placeholder="Amount" required>

            <input type="date" id="date" required>

            <input list="categories" id="category" placeholder="Category" required>
            <datalist id="categories">
                <option value="Food">
                <option value="Travel">
                <option value="Shopping">
                <option value="Salary">
                <option value="Bills">
            </datalist>

            <select id="type">
                <option value="income">Income</option>
                <option value="expense">Expense</option>
            </select>

            <button type="submit">Add</button>
        </form>

        <h3>Financial Tips</h3>
        <ol>
            <li>Track daily expenses</li>
            <li>Save at least 20% income</li>
            <li>Avoid unnecessary subscriptions</li>
        </ol>
    </article>
```

```
    <article class="table-box">
        <h2>Transactions</h2>

        <table>
            <thead>
                <tr>
                    <th>Date</th>
                    <th>Title</th>
                    <th>Category</th>
                    <th>Type</th>
                    <th>Amount</th>
                </tr>
            </thead>
            <tbody id="tableBody">
            </tbody>
        </table>

        <div class="summary">
            <p>Total Balance: ₹ <span id="balance">0</span></p>
            <progress id="savingBar" value="0" max="100"></progress>
            <p>Expense Ratio</p>
            <meter id="expenseMeter" min="0" max="100" value="0"></meter>
        </div>

    </article>

</section>

<script src="finance.js"></script>
</body>
</html>
```

File-2:finance.css

```
body {
    margin: 0;
    font-family: Arial;
    background: #eef2f7;
}
```

```
header {
    background: #1e293b;
    color: white;
    padding: 15px;
    text-align: center;
}

.layout {
    display: flex;
    gap: 20px;
    padding: 20px;
}

.form-box, .table-box {
    background: white;
    padding: 15px;
    border: 2px solid #ccc;
    flex: 1;
}

input, select, button {
    display: block;
    margin: 8px 0;
    padding: 6px;
    width: 100%;
}

button {
    background: #2563eb;
    color: white;
    border: none;
}

button:hover {
    background: #1d4ed8;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th, td {
    border: 1px solid #ddd;
```

```
    padding: 6px;
    text-align: center;
}

.incomeRow {
    background: #dcfce7;
}

.expenseRow {
    background: #fee2e2;
}

.summary {
    margin-top: 15px;
}

@media(max-width: 768px) {
    .layout {
        flex-direction: column;
    }
}
```

File-3:finance.js

```
let transactions = JSON.parse(localStorage.getItem("financeData")) || [];

const form = document.getElementById("txnForm");
const tableBody = document.getElementById("tableBody");

form.addEventListener("submit", function(e) {
    e.preventDefault();
    addTransaction();
});

function addTransaction() {

    let title = document.getElementById("title").value;
    let amount = parseFloat(document.getElementById("amount").value);
    let date = document.getElementById("date").value;
    let category = document.getElementById("category").value;
    let type = document.getElementById("type").value;
```

```
    if (amount <= 0) {
        alert("Amount must be positive");
        return;
    }

    let newTxn = {
        title,
        amount,
        date,
        category,
        type
    };

    transactions.push(newTxn);

    localStorage.setItem("financeData", JSON.stringify(transactions));

    updateTable();
    calculateTotal();

    form.reset();
}

function updateTable() {
    tableBody.innerHTML = "";

    transactions.forEach(function(txn) {

        let row = document.createElement("tr");

        if (txn.type === "income") {
            row.classList.add("incomeRow");
        } else {
            row.classList.add("expenseRow");
        }

        row.innerHTML = `
            <td>${txn.date}</td>
            <td>${txn.title}</td>
            <td>${txn.category}</td>
            <td>${txn.type}</td>
            <td>₹ ${txn.amount}</td>
        `;
```

```
        tableBody.appendChild(row);
    });
}

function calculateTotal() {

    let income = 0;
    let expense = 0;

    transactions.forEach(function(txn) {
        if (txn.type === "income") {
            income += txn.amount;
        } else {
            expense += txn.amount;
        }
    });

    let balance = income - expense;

    document.getElementById("balance").innerText = balance;

    let savingPercent = income > 0 ? (balance / income) * 100 : 0;
    document.getElementById("savingBar").value = savingPercent;

    let expenseRatio = income > 0 ? (expense / income) * 100 : 0;
    document.getElementById("expenseMeter").value = expenseRatio;
}

updateTable();
calculateTotal();
```

Project 3 — "Multi-Page E-Learning Platform"

File-1:dashboard.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Learning System - Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<header>
  <h1>My Learning System</h1>
  <nav>
    <a href="dashboard.html">Dashboard</a>
    <a href="courses.html">Courses</a>
    <a href="quiz.html">Quiz</a>
    <a href="profile.html">Profile</a>
  </nav>
</header>

<section class="gridArea">
  <div class="card">
    <h3>Total Courses</h3>
    <p id="totalC"></p>
  </div>

  <div class="card">
    <h3>Completed</h3>
    <p id="doneC"></p>
  </div>

  <div class="card">
    <h3>Progress</h3>
    <progress id="courseProg" value="0" max="100"></progress>
  </div>
</section>

<script src="script.js"></script>
</body>
</html>
```

File-2:courses.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Courses</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<nav>
  <a href="dashboard.html">Dashboard</a> >
  <a href="courses.html">Courses</a>
```

```
</nav>

<h2>Course List</h2>

<table>
  <thead>
    <tr>
      <th>Course Name</th>
      <th>Status</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody id="courseBody"></tbody>
</table>

<h3>Lesson Steps</h3>
<ol>
  <li>Study Topic</li>
  <li>Practice</li>
  <li>Take Quiz</li>
</ol>

<script src="script.js"></script>
</body>
</html>
```

File-3:quiz.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Quiz</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<nav>
  <a href="dashboard.html">Dashboard</a> >
  <a href="quiz.html">Quiz</a>
</nav>

<section id="quizBox">
  <p>Loading Quiz...</p>
</section>
```

```
<button onclick="submitQuiz()">Submit Quiz</button>

<section id="resultArea"></section>

<script src="script.js"></script>
</body>
</html>
```

File-4:profile.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Profile</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<nav>
  <a href="dashboard.html">Dashboard</a> >
  <a href="profile.html">Profile</a>
</nav>

<h2>Your Performance</h2>
<p>Last Quiz Score: <span id="lastScore"></span></p>

<script src="script.js"></script>
</body>
</html>
```

File-5:style.css

```
body {
  margin: 0;
  font-family: Arial;
  background: #f2f4f8;
}

header {
  background: #1e293b;
  color: white;
  padding: 10px;
```

```
}

nav a {
  margin-right: 10px;
  text-decoration: none;
  color: darkblue;
}

.gridArea {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 15px;
  padding: 20px;
}

.card {
  background: white;
  padding: 15px;
  border: 1px solid #ccc;
}

.card:hover {
  background: #e0f2fe;
}

table {
  width: 100%;
  margin-top: 15px;
  border-collapse: collapse;
}

th, td {
  border: 1px solid #ccc;
  padding: 8px;
  text-align: center;
}

tr:nth-child(even) {
  background: #f9fafb;
}

button:hover {
  background: lightgreen;
}
```

```
@media(max-width:700px){
  .gridArea {
    grid-template-columns: 1fr;
  }
}
```

File-6:script.js

```
// DATA
let defaultCourses = [
  {id:1, name:"HTML Basics", done:false},
  {id:2, name:"CSS Design", done:false},
  {id:3, name:"JavaScript Core", done:false}
];

let courses = JSON.parse(localStorage.getItem("courseStore")) || defaultCourses;

let questions = [
  {q:"HTML used for?", options:["Structure","Database","Server"], ans:0},
  {q:"CSS is for?", options:["Logic","Styling","Storage"], ans:1}
];

//  SAVE
function saveCourses(){
  localStorage.setItem("courseStore", JSON.stringify(courses));
}

// DASHBOARD
if(document.getElementById("totalC")){
  document.getElementById("totalC").innerText = courses.length;

  let completed = courses.filter(c=>c.done).length;
  document.getElementById("doneC").innerText = completed;

  document.getElementById("courseProg").value =
    (completed/courses.length)*100;
}

// COURSES PAGE
if(document.getElementById("courseBody")){
  let body = document.getElementById("courseBody");
  body.innerHTML = "";
```

```
  courses.forEach(c=>{
    let row = document.createElement("tr");

    row.innerHTML = `
      <td>${c.name}</td>
      <td>${c.done ? "Completed" : "Pending"}</td>
      <td><button onclick="markComplete(${c.id})">Done</button></td>
    `;

    body.appendChild(row);
  });
}

function markComplete(id){
  courses = courses.map(c=>{
  if(c.id === id){
      c.done = true;
    }
    return c;
  });
  saveCourses();
  location.reload();
}

//  QUIZ PAGE
async function loadQuiz(){

  if(!document.getElementById("quizBox")) return;

  await new Promise(resolve => setTimeout(resolve,1500));

  let box = document.getElementById("quizBox");
  box.innerHTML = "";

  questions.forEach((item,i)=>{
    let div = document.createElement("div");
    div.innerHTML = `<p>${item.q}</p>`;

    item.options.forEach((opt,index)=>{
      div.innerHTML += `
        <input type="radio" name="q${i}" value="${index}"
        onchange="answerChanged()"> ${opt}<br>
      `;
    });
```

```
    box.appendChild(div);
  });
}

function answerChanged(){
  console.log("Option selected");
}

function submitQuiz(){

  let score = 0;

  questions.forEach((q,i)=>{
    let selected = document.querySelector(`input[name="q${i}"]:checked`);
    if(selected && parseInt(selected.value) === q.ans){
      score++;
    }
  });

  let percent = (score/questions.length)*100;

  let result;

  if(percent >= 50){
    result = "Pass";
  } else {
    result = "Fail";
  }

  switch(result){
    case "Pass":
      result += " - Well Done";
      break;
    case "Fail":
      result += " - Try Again";
      break;
  }

  localStorage.setItem("quizScore", percent);

  document.getElementById("resultArea").innerHTML =
    `<h3>${percent}%</h3><p>${result}</p>`;
}
```

```
loadQuiz();

//PROFILE
if(document.getElementById("lastScore")){
  let score = localStorage.getItem("quizScore");
  document.getElementById("lastScore").innerText =
  score || "No Quiz Attempted";
}
```

Project 4 — "Geo-Based Task Manager with Async Storage Simulation"

File-1:index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Geo Task Manager</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <h1>Field Task Tracker</h1>
</header>

<main class="layout">

    <!-- Form Section -->
    <section class="form-area">
        <h2>Create Task</h2>

        <form id="taskForm">
            <input type="text" id="taskTitle" placeholder="Task title" required>

            <input type="date" id="taskDate" required>

            <input type="number" id="taskNumber" placeholder="Estimated Hours"
min="1">

            <input list="priorityList" id="taskPriority" placeholder="Select priority"
required>
            <datalist id="priorityList">
                <option value="Low">
```

```
            <option value="Medium">
            <option value="High">
        </datalist>

        <label>
            <input type="checkbox" id="taskDone">
            Completed
        </label>

        <button type="submit">Add Task</button>
    </form>
</section>

<!-- Table Section -->
<section class="table-area">
    <h2>Task List</h2>

    <table>
        <thead>
            <tr>
                <th>Title</th>
                <th>Priority</th>
                <th>Location</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody id="taskBody"></tbody>
    </table>

    <div class="stats">
        <label>Completion:</label>
        <progress id="taskProgress" value="0" max="100"></progress>

        <label>High Priority Load:</label>
        <meter id="priorityMeter" min="0" max="10" value="0"></meter>
    </div>
</section>

</main>

<script src="app.js"></script>
</body>
</html>
```

File-2:style.css

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f4f6f8;
}

header {
    background: #1e3a8a;
    color: white;
    padding: 15px;
    text-align: center;
}

.layout {
    display: grid;
    grid-template-columns: 1fr 2fr;
    gap: 20px;
    padding: 20px;
}

.form-area, .table-area {
    background: white;
    padding: 15px;
    border-radius: 8px;
}

/* Flexbox inside form */
form {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

button {
    background: #2563eb;
    color: white;
    border: none;
    padding: 8px;
    cursor: pointer;
}
```

```
button:hover {
    background: #1d4ed8;
}

/* Advanced selector */
table tbody tr:nth-child(even) {
    background: #f1f5f9;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th, td {
    padding: 8px;
    text-align: left;
}

/* Responsive */
@media (max-width: 768px) {
    .layout {
        grid-template-columns: 1fr;
    }
}
```

File-3:app.js

```
}

let taskArray = JSON.parse(localStorage.getItem("geoTasks")) || [];

// Load saved tasks
document.addEventListener("DOMContentLoaded", () => {
    renderTasks();
});

document.getElementById("taskForm").addEventListener("submit", async function(e)
{
    e.preventDefault();

    const title = taskTitle.value;
    const priority = taskPriority.value;
```

```
    const completed = taskDone.checked;

    try {
        const location = await getLocation();
        await fakeServerSave();

        const taskObj = {
            id: Date.now(),
            title,
            priority,
            location,
            completed
        };

        taskArray.push(taskObj);
        localStorage.setItem("geoTasks", JSON.stringify(taskArray));

        renderTasks();
        this.reset();

    } catch (err) {
        alert("Error: " + err);
    }
});

// Geolocation
function getLocation() {
    return new Promise((resolve, reject) => {
        navigator.geolocation.getCurrentPosition(
            pos => {
                resolve(
                    pos.coords.latitude.toFixed(2) + "," +
                    pos.coords.longitude.toFixed(2)
                );
            },
            err => {
                switch(err.code) {
                    case 1: reject("Permission denied"); break;
                    case 2: reject("Position unavailable"); break;
                    case 3: reject("Timeout"); break;
                    default: reject("Unknown error");
                }
            }
        );
```

```
    });
}

// Fake async server save
function fakeServerSave() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            Math.random() > 0.1 ? resolve("Saved") : reject("Server failed");
        }, 1000);
    });
}

// Render table
function renderTasks() {
    const tbody = document.getElementById("taskBody");
    tbody.innerHTML = "";

    taskArray.forEach(task => {
        const row = `
            <tr>
                <td>${task.title}</td>
                <td>${task.priority}</td>
                <td>${task.location}</td>
                <td>${task.completed ? "Done" : "Pending"}</td>
            </tr>
        `;
        tbody.innerHTML += row;
    });

    updateStats();
}

// Update progress + meter
function updateStats() {
    const total = taskArray.length;

    const completedTasks = taskArray.filter(t => t.completed).length;
    const highPriority = taskArray.filter(t => t.priority === "High").length;

    const percent = total === 0 ? 0 : (completedTasks / total) * 100;

    document.getElementById("taskProgress").value = percent;
    document.getElementById("priorityMeter").value = highPriority;
}
```

```
// Extra array usage example
function findTaskById(id) {
    return taskArray.find(t => t.id === id);
}
```